Advanced Orchestration of Wi-Fi Direct Manipulation and Command-Line Compilation in Android Systems

The Android operating system relies on a highly sophisticated networking stack capable of executing concurrent, complex topologies. Among the most challenging architectural requirements for an Android application is the simultaneous orchestration of a Wi-Fi Infrastructure connection—acting as a Station (STA) to route public internet traffic—and a direct, local peer-to-peer (P2P) network. This dual-interface concurrency allows a device to share data securely and directly with a nearby peer without disrupting its own external network connectivity. However, establishing this architecture introduces severe complexities when developers must programmatically dictate the broadcasted name of the Wi-Fi Direct interface from within an application's MainActivity. Android's evolving security model strictly regulates access to low-level hardware identifiers, necessitating a nuanced approach to network naming conventions. Compounding this network engineering challenge is the requirement to bypass conventional build systems. Extricating an application from the Gradle ecosystem and compiling it purely through a command-line toolchain demands an exhaustive understanding of Android's compilation binaries. This report provides a comprehensive architectural analysis of manipulating Wi-Fi Direct network configurations, managing dual-network interface socket binding, and manually compiling the resulting application via the Android Asset Packaging Tool (AAPT2), the Dalvik Executable compiler (D8), and cryptographic signing utilities.

Mechanisms for Wi-Fi Direct Naming and Group Orchestration

The Wi-Fi Peer-to-Peer (P2P) framework empowers Android devices to discover, negotiate, and establish direct high-bandwidth connections using WPA2-encrypted access points, bypassing the need for an intermediate router[1]. By default, the Service Set Identifier (SSID) broadcasted during this discovery phase is deeply coupled to the global system device name configured by the user in the core operating system settings. Programmatically modifying this identifier specifically for the scope of a single application's MainActivity requires circumventing or leveraging specific Android APIs depending on the target API level.

Legacy Device Naming via Java Reflection (Pre-Android 10)

In earlier iterations of the Android Software Development Kit (SDK), the WifiP2pManager class did not expose a public method for altering the device's P2P name. The core framework source code implemented a setDeviceName() method; however, it was decorated with the @hide annotation, intentionally omitting it from the public android.jar stub used for compilation[2]. To dynamically change the name broadcasted to peers, developers relied heavily on Java reflection to breach this encapsulation at runtime.

The execution of this reflection within the MainActivity necessitated a precise sequence of operations. The WifiP2pManager instance, obtained via getSystemService(Context.WIFI_P2P_SERVICE), acts as the primary conduit to the Wi-Fi hardware[4]. The developer was required to construct an array of Class objects representing the exact parameter signature of the hidden method: the WifiP2pManager.Channel, a String denoting the desired device name, and an instance of WifiP2pManager.ActionListener to capture asynchronous success or failure callbacks[6].

By invoking manager.getClass().getMethod("setDeviceName", paramTypes) and subsequently calling setAccessible(true), the runtime enforced visibility over the hidden API[2]. The invoke() method was then executed against the manager instance, passing the initialized channel, the truncated name string (which the framework strictly limits to 32 characters), and the callback implementation[6]. While this effectively transformed the device name for subsequent peer discovery sequences, the technique has been progressively deprecated. Starting with Android 9 (API level 28), the introduction of non-SDK interface restrictions aggressively penalized reflection on undocumented APIs[3]. By Android 11 (API level 30), and definitively in Android 14 (API level 34), executing this reflection triggers a severe SecurityException. The operating system intercepts the call, throwing a permission violation indicating the application lacks the NETWORK_SETTING, NETWORK_STACK, or OVERRIDE_WIFI_CONFIG system-level permissions[8]. Consequently, this methodology is entirely non-viable for modern production environments.

Autonomous Group Ownership and the Configuration Builder (Android 10+)

Recognizing the necessity for applications to explicitly dictate connection parameters, Android 10 (API level 29) introduced the WifiP2pConfig.Builder class. This enhancement fundamentally shifted the paradigm from dynamically hacking the global device name to explicitly configuring the attributes of an autonomous Group Owner (GO)[9].

When an Android device assumes the role of an autonomous GO, it bypasses the standard peer negotiation phase and instantly broadcasts an access point tailored to the developer's specifications[12]. The WifiP2pConfig.Builder facilitates this by exposing the setNetworkName() and setPassphrase() methods, enabling the MainActivity to securely define the WPA2-PSK credentials[9]. Furthermore, the setGroupOperatingBand() method allows the application to dictate the radio frequency (e.g., WifiP2pConfig.GROUP_OWNER_BAND_5GHZ), a critical feature for establishing low-latency, high-throughput connections independent of the standard 2.4GHz spectrum congestion[9].

A strict protocol constraint accompanies this modern API. The string supplied to setNetworkName() is subjected to internal validation and must unconditionally begin with the prefix DIRECT-xy, where x and y are alphanumeric characters[13]. This prefix is a standardized IEEE 802.11 convention for Wi-Fi Direct networks, and any attempt to bypass it using standard SDK tools will result in an IllegalArgumentException. Once the configuration object is instantiated via the .build() command, the MainActivity invokes manager.createGroup(channel, config, listener)[11]. This cleanly forces the hardware to broadcast the explicitly defined, prefixed

SSID, establishing an endpoint to which external peers can connect seamlessly using standard socket configurations.

Strategy	Applicable API Levels	Mechanism	Constraint
setDeviceName Reflection	API 14 - 28	Reflective invocation of hidden WifiP2pManager method.	Blocked by non-SDK interface restrictions; throws SecurityException on Android 14.
WifiP2pConfig.Builder	API 29+	Formal instantiation of autonomous Group Owner properties.	SSID must adhere strictly to the DIRECT-xy prefix formatting.
SoftApConfiguration Reflection	API 33+	Bypass P2P entirely via local-only hotspot hidden APIs.	Requires NEARBY_WIFI_DEVICES permission and complex reflection arrays.

The LocalOnlyHotspot Alternative for Custom SSIDs

In architectural scenarios where the DIRECT- prefix is fundamentally incompatible with legacy hardware peers or specific application requirements, developers must pivot from Wi-Fi P2P to the Soft Access Point (SoftAP) infrastructure. The startLocalOnlyHotspot() API, introduced in Android 8.0 (API level 26), provisions a standalone Wi-Fi network that explicitly forbids routing traffic to the external internet[16]. By design, this public API intentionally generates a randomized SSID and passphrase, returning these transient credentials to the application via the LocalOnlyHotspotReservation object to prevent static tracking[18].

To enforce a specific, static SSID and passphrase without the DIRECT- prefix, advanced integration requires subverting the SoftApConfiguration framework. Android 13 (API level 33) introduced underlying architectural changes that allow the startLocalOnlyHotspot method to accept a SoftApConfiguration object as a parameter, though this specific overloaded signature remains hidden from the public SDK[19].

To utilize this, the MainActivity must employ reflection to load the hidden android.net.wifi.SoftApConfiguration$Builder class using Class.forName(). The developer instantiates this hidden builder and invokes its setSsid() and setPassphrase() methods[21]. The passphrase injection requires an additional parameter defining the security protocol, typically

mapping to SoftApConfiguration.SECURITY_TYPE_WPA2_PSK[21]. After successfully reflecting the .build() method to yield a valid configuration object, the application must execute a secondary reflection against the WifiManager. By mapping the parameter array to SoftApConfiguration.class, Executor.class, and WifiManager.LocalOnlyHotspotCallback.class, the application can invoke the hidden local-only hotspot daemon[19]. While this technique successfully provisions a fully customized SSID, it navigates a fragile perimeter of Android's security architecture and requires meticulous exception handling.

Multi-Interface Concurrency and Socket Binding

Manipulating the Wi-Fi P2P or SoftAP configuration addresses only the physical layer broadcast. The greater challenge lies in maintaining dual-interface concurrency. The application must retain an active Station (STA) connection to an external router providing internet access, while simultaneously operating the newly minted local access point.

Dual-Band Simultaneous (DBS) Hardware Constraints

The ability of an Android device to operate as a concurrent STA and Access Point (AP/P2P) is strictly gated by the device's physical hardware and its corresponding Hardware Abstraction Layer (HAL)[23]. The internal Wi-Fi chip must possess multiple independent radio chains (MACs) capable of operating asynchronously. Within the Android framework, the BoardConfig-common.mk file must declare the WIFI_HIDL_FEATURE_DUAL_INTERFACE := true build flag[23].

When concurrency is actively triggered, the device brings up the wlan0 interface for the primary internet connection, and simultaneously spins up the p2p-wlan0-0 (or wlan1) interface for the direct peer connection[23]. To mitigate extreme latency and packet collisions, the Wi-Fi firmware utilizes Channel Switch Avoidance (CSA) to dynamically migrate the local access point. The firmware either aligns both interfaces onto the exact same frequency channel to utilize time-division multiplexing, or forcibly separates them across non-overlapping bands (e.g., maintaining the STA connection on a 2.4GHz channel while projecting the P2P group on a 5GHz channel)[23].

Process-Wide Routing and Network Callbacks

When a device maintains two distinct network interfaces, the Android routing table is faced with a path-resolution conflict. By default, standard socket creations within the Dalvik Virtual Machine will route packets through the default gateway, which is inextricably linked to the interface possessing internet capability (the STA interface). If the MainActivity attempts to open a Socket or DatagramSocket to communicate with the connected peer, those packets will misroute into the external internet router, resulting in Host Unreachable or connection timeout exceptions[26].

To direct traffic accurately to the P2P interface, the developer must employ the ConnectivityManager and explicitly bind the application's sockets to the peer-to-peer network object. This requires constructing a highly specific NetworkRequest.

NetworkRequest API	Required Parameter	Architectural Purpose

clearCapabilities()	None	Strips default system constraints (like the requirement for NET_CAPABILITY_INTERNET), ensuring local-only networks are not filtered out.
addTransportType()	NetworkCapabilities.TRANSPORT_WIFI	Restricts the network listener to Wi-Fi hardware exclusively, ignoring Cellular or Ethernet interfaces.
addCapability()	NetworkCapabilities.NET_CAPABILITY_WIFI_P2P	Instructs the framework to isolate and select the dynamically created Wi-Fi Direct interface.

The MainActivity executes mConnectivityManager.registerNetworkCallback(request, networkCallback). Once the P2P negotiation is successful and the interface acquires an IP address, the system triggers the onAvailable(Network network) override[26]. Within this callback, the application achieves deterministic routing by invoking mConnectivityManager.bindProcessToNetwork(network)[26]. This global command overwrites the default routing table for the entire process space, forcing all newly instantiated sockets to traverse the P2P interface.
If global binding interferes with background threads requiring cloud connectivity, the application should bypass bindProcessToNetwork and instead rely on targeted binding. By invoking network.bindSocket(socket) immediately after opening a new DatagramSocket or TCP Socket, the specific connection is hard-routed to the local peer, preserving the application's simultaneous external internet access on all unbound sockets[27].

Low-Level Execution: The Command-Line Build Pipeline

Integrating the complex Java logic required for network binding, reflection, and broadcast receivers constitutes only the source-code layer. The requirement to compile this Android application entirely independent of Gradle or the Android Studio Integrated Development Environment (IDE) mandates manual interaction with the Android SDK build tools. The compilation pipeline involves raw asset packaging, bytecode translation, memory alignment, and cryptographic validation[29].

The primary advantage of a pure command-line toolchain is absolute deterministic control over the generated binary, removing the obfuscation of dependency resolution algorithms and automated manifest merging inherent in modern Gradle implementations.

Phase 1: Resource Indexing and Compilation (AAPT2)

Android operating systems rely on highly optimized binary representations of XML files, manifests, and graphical assets to ensure rapid memory mapping at runtime. The Android Asset Packaging Tool version 2 (AAPT2) handles the translation of human-readable resources into this binary format[29]. This process is strictly divided into a compile phase and a link phase[29]. During the compilation step, the developer points AAPT2 to the project's res/ directory. AAPT2 parses every strings.xml, layout.xml, and drawable asset, generating intermediate .flat files[29]. These intermediate binaries allow for incremental compilation, saving vast amounts of processing time in larger projects[29].

Following compilation, the link phase is executed. AAPT2 takes all generated .flat files, the raw AndroidManifest.xml, and the platform-specific android.jar (representing the framework dependencies) to synthesize the application's resource table (resources.arsc)[29]. Crucially, the link command must include the --java flag, which forces AAPT2 to emit the R.java file. This generated source code serves as a static registry mapping human-readable XML IDs to compiled memory addresses (e.g., 0x7f040001), allowing the developer's MainActivity to reference user interface elements directly[30]. The output of this phase is an unsigned, unaligned base.apk stripped of any compiled Java logic.

Phase 2: Bytecode Synthesis and Dalvik Translation

With the R.java file generated, the pure Java compilation begins. The standard JDK javac compiler is invoked, taking both the MainActivity.java and the newly minted R.java as inputs. The compiler's classpath must be explicitly pointed to the android.jar matching the target API level to resolve Android-specific imports like android.net.wifi.p2p.WifiP2pManager[30]. The resulting output is a directory populated with standard JVM .class bytecode files.

However, the Android Runtime (ART) does not execute standard Java bytecode; it relies on the Dalvik Executable (DEX) format, which is heavily optimized for low-memory, battery-constrained mobile processors. The d8 tool executes this transformation[30].

When d8 is invoked, it digests the compiled .class files and the android.jar. During this translation, d8 executes a complex procedure known as desugaring. Android's framework historically lacked native support for advanced Java 8+ features, such as lambda expressions and default interface methods. Desugaring intercepts these modern syntactic elements and rewrites them into compatible, legacy bytecode structures that can be safely executed on older Android API levels[33]. The terminal output of this operation is a single classes.dex file containing the totality of the application's executable logic.

Compilation Stage	Input Artifacts	SDK Tool	Output Artifact	Architectural Function

Resource Compile	XML Layouts, Values, PNGs	aapt2 compile	.flat Binaries	Translates human-readable resources into incremental binary formats.
Resource Link	.flat files, Manifest	aapt2 link	base.apk, R.java	Synthesizes resources.arsc, embeds the manifest, maps memory IDs.
Bytecode Compile	.java Source, R.java	javac	.class Files	Standard JVM bytecode compilation requiring android.jar in the classpath.
Dalvik Translation	.class Files	d8	classes.dex	Desugars Java 8 constructs and optimizes bytecode for the Android Runtime (ART).

Phase 3: Archive Assembly and Memory Alignment

The classes.dex file must now be physically injected into the base.apk archive generated during the AAPT2 link phase. Utilizing the standard UNIX zip utility, the .dex file is appended to the root directory of the APK. It is a critical architectural requirement that the .dex file remains uncompressed during this injection; compressing the bytecode prevents the Android operating system from directly reading it, forcing an expensive extraction process into local storage during installation[34].

Following assembly, the APK must undergo memory alignment. The zipalign tool parses the archive and adjusts the byte offset of every uncompressed file (such as raw audio assets, native libraries, and the newly injected .dex file) to ensure they start on a precise 4-byte boundary relative to the beginning of the file[36]. This seemingly minor adjustment yields massive performance dividends. When the application is launched, the Android kernel can memory-map

(mmap) the aligned assets directly from the APK into RAM, entirely bypassing the need to copy data into secondary buffers. This preserves the device's heap space and drastically reduces the application's memory footprint[36].

Phase 4: Cryptographic Validation and Deployment

The Android security paradigm strictly mandates that all deployable packages carry a valid cryptographic signature proving their origin and ensuring their structural integrity has not been tampered with post-compilation[37]. The developer must first synthesize a public/private key pair encapsulated within a Java Keystore (.jks) using the keytool command[38].

Once the keystore is provisioned, the apksigner utility evaluates the aligned APK[40]. Unlike deprecated legacy signing methods that merely hashed individual files (v1 signatures), the modern apksigner injects an APK Signature Scheme block (v2 or v3) directly into the binary structure of the file, immediately preceding the Central Directory. This computes a cryptographic hash over the entire contiguous file sequence, guaranteeing that even a single byte modification to the application's metadata or resources will instantly invalidate the signature and trigger an installation failure on the host device[41]. Once the apksigner verify command returns a successful validation, the application is structurally complete and ready for deployment via the Android Debug Bridge (adb).

The MainActivity Integration: Permissions and Callbacks

The synthesis of the aforementioned network logic and build parameters culminates in the architecture of the MainActivity. Before the Wi-Fi hardware can be manipulated, the application must navigate Android's increasingly stringent runtime permission model[43].

Historically, discovering nearby devices required the application to secure the ACCESS_FINE_LOCATION permission[44]. The system rationale was that an application capable of reading the Basic Service Set Identifiers (BSSIDs) of nearby hardware could theoretically triangulate the physical geographic location of the user[44]. Consequently, legacy codebases required developers to prompt the user for high-accuracy GPS tracking simply to send a local file.

Android 13 (API level 33) revolutionized this architecture by introducing the NEARBY_WIFI_DEVICES permission[44]. This granular runtime permission explicitly decoupled local network operations from physical geolocation. To satisfy the operating system's security audit, the developer must embed android:usesPermissionFlags="neverForLocation" alongside the permission declaration in the AndroidManifest.xml[44]. Within the MainActivity.onCreate() lifecycle method, the application utilizes ContextCompat.checkSelfPermission() to verify the presence of this token, failing over to ActivityCompat.requestPermissions() if the user has not yet granted access[43].

Orchestrating Asynchronous State Changes

Wi-Fi direct operations do not execute sequentially on the main thread; they rely entirely on an

asynchronous intent-driven architecture[4]. The MainActivity must define a custom BroadcastReceiver dedicated to capturing state transitions emitted by the WifiP2pManager. Within the onResume() method, an IntentFilter is registered containing WIFI_P2P_STATE_CHANGED_ACTION (verifying hardware availability) and WIFI_P2P_CONNECTION_CHANGED_ACTION (monitoring peer connectivity)[4]. When the application broadcasts its configured SSID using the WifiP2pConfig.Builder and a peer successfully authenticates, the operating system dispatches the connection intent. The BroadcastReceiver intercepts this intent and immediately invokes manager.requestConnectionInfo(channel, connectionListener)[4].
This callback yields a WifiP2pInfo object, a critical data structure containing the groupOwnerAddress[5]. The application extracts the IPv4 address string from this object via .getHostAddress(). Empowered with the peer's exact IP address, and fortified by the ConnectivityManager.bindProcessToNetwork() command guaranteeing the traffic bypasses the external internet router, the application can securely instantiate a Socket mapped to the designated port, ensuring high-speed data transfer across the concurrent Wi-Fi interface. Through the meticulous alignment of permission strategies, intent-driven asynchronous callbacks, multi-interface routing constraints, and low-level command-line compilation, the application achieves a highly optimized, fully deterministic implementation of peer-to-peer networking entirely decoupled from conventional automated build environments.

Works cited

1. Create P2P connections with Wi-Fi Direct - Android Developers, https://developer.android.com/develop/connectivity/wifi/wifi-direct
2. Android rename device's name for wifi-direct - Stack Overflow, https://stackoverflow.com/questions/27315198/android-rename-devices-name-for-wifi-direct
3. How to WifiP2pDevice.deviceName for current device? - Stack Overflow, https://stackoverflow.com/questions/53607516/how-to-wifip2pdevice-devicename-for-current-device
4. Wi-Fi Direct (peer-to-peer or P2P) overview | Connectivity - Android Developers, https://developer.android.com/develop/connectivity/wifi/wifip2p
5. android.net.wifi.p2p.WifiP2pManager - Documentation - HCL Software Open Source, http://opensource.hcltechsw.com/volt-mx-native-function-docs/Android/android.net.wifi.p2p-Android-10.0/#!/api/android.net.wifi.p2p.WifiP2pManager
6. Change the Wifi Direct name in android - Stack Overflow, https://stackoverflow.com/questions/27701236/change-the-wifi-direct-name-in-android
7. Android naming a P2P device - java - Stack Overflow, https://stackoverflow.com/questions/28664108/android-naming-a-p2p-device
8. Cannot use WifiP2pManager.setDeviceName on Android 11 (Wi-Fi Direct) - Stack Overflow, https://stackoverflow.com/questions/67398342/cannot-use-wifip2pmanager-setdev

icename-on-android-11-wi-fi-direct
9. WifiP2pConfig.Builder | API reference - Android Developers, https://developer.android.com/reference/kotlin/android/net/wifi/p2p/WifiP2pConfig.Builder
10. WifiP2pConfig.Builder Class (Android.Net.Wifi.P2p) | Microsoft Learn, https://learn.microsoft.com/en-us/dotnet/api/android.net.wifi.p2p.wifip2pconfig.builder?view=net-android-35.0
11. Android 10 features and APIs, https://developer.android.com/about/versions/10/features
12. How to create an autonomous GO for Wifi Direct in Android with dedicated ssid and passphrase? - Stack Overflow, https://stackoverflow.com/questions/19018141/how-to-create-an-autonomous-go-for-wifi-direct-in-android-with-dedicated-ssid-an
13. android.net.wifi.p2p.WifiP2pConfig.Builder - Documentation - HCL Software Open Source, http://opensource.hcltechsw.com/volt-mx-native-function-docs/Android/android.net.wifi.p2p-Android-10.0/#!/api/android.net.wifi.p2p.WifiP2pConfig.Builder
14. WifiP2pConfig.Builder | API reference - Android Developers, https://developer.android.com/reference/android/net/wifi/p2p/WifiP2pConfig.Builder
15. How do I force android WiFi Direct group owner to use 2.4 GHz instead of 5 GHz, https://stackoverflow.com/questions/61140789/how-do-i-force-android-wifi-direct-group-owner-to-use-2-4-ghz-instead-of-5-ghz
16. turning on Hotspot | B4X Programming Forum, https://www.b4x.com/android/forum/threads/turning-on-hotspot.143356/
17. Use a local-only Wi-Fi hotspot | Connectivity - Android Developers, https://developer.android.com/develop/connectivity/wifi/localonlyhotspot
18. Change local-only Wi-Fi hotspot's SSID and password in Android Oreo 8.x - Stack Overflow, https://stackoverflow.com/questions/47700717/change-local-only-wi-fi-hotspots-ssid-and-password-in-android-oreo-8-x
19. "Manually configure SSID and passphrase for LocalOnlyHotspot in android studio?" - Stack Overflow, https://stackoverflow.com/questions/68642244/manually-configure-ssid-and-passphrase-for-localonlyhotspot-in-android-studio
20. Configuring Android's LocalOnlyHotspot (5Ghz, defining SSID, BSSID, and more) - Medium, https://medium.com/@mike_21858/configuring-androids-localonlyhotspot-5ghz-defining-ssid-bssid-and-more-ef4e4975e7b4
21. Android 13+ LocalOnlyHotspot custom config - GitHub Gist, https://gist.github.com/spieglt/09027eefe098af6a6207181ad3aeabf7
22. SoftApConfiguration.Builder | API reference - Android Developers, https://developer.android.com/reference/android/net/wifi/SoftApConfiguration.Builder
23. Wi-Fi STA/AP concurrency | Android Open Source Project,

https://source.android.com/docs/core/connect/wifi-sta-ap-concurrency
24. Mastering WiFi Concurrency: WiFi HAL, STA/STA, and STA/AP Explained | by Akash Das, https://medium.com/@akashdasinn89/mastering-wifi-concurrency-wifi-hal-sta-sta-and-sta-ap-explained-a940f288c217
25. Connect Android smartphone with Wi-Fi Direct to a Raspberry Pi, https://raspberrypi.stackexchange.com/questions/117238/connect-android-smartphone-with-wi-fi-direct-to-a-raspberry-pi
26. How to bindProcessToNetwork for a Wi-Fi P2P network? - Stack Overflow, https://stackoverflow.com/questions/79194019/how-to-bindprocesstonetwork-for-a-wi-fi-p2p-network
27. Network | API reference - Android Developers, https://developer.android.com/reference/kotlin/android/net/Network
28. Network | API reference - Android Developers, https://developer.android.com/reference/android/net/Network
29. AAPT2 | Android Studio, https://developer.android.com/tools/aapt2
30. Building an Android App Without Gradle or Android Studio | by Nolifek - Medium, https://medium.com/@kacperkotlowski/building-an-android-app-without-gradle-or-android-studio-49732e58ef0b
31. How can I build an Android apk without Gradle on the command line? - Stack Overflow, https://stackoverflow.com/questions/41132753/how-can-i-build-an-android-apk-without-gradle-on-the-command-line
32. From code to pixels — Android's approach - Esmund - Medium, https://esmund.medium.com/from-code-to-pixels-androids-approach-3b8ba65dd276
33. d8 | Android Studio, https://developer.android.com/tools/d8
34. Android Build with command-line tools - Stack Overflow, https://stackoverflow.com/questions/69566622/android-build-with-command-line-tools
35. Android CLI APK Builder for arm64 - Build APKs from command line without Android Studio/Gradle. Box64 translates native x86_64 tools (aapt2, zipalign) while Java tools (d8, apksigner) run natively. · GitHub, https://gist.github.com/felix021/e7179596244ee81852c646f904adddf6
36. Build your app from the command line | Android Studio, https://developer.android.com/build/building-cmdline
37. Sign your app | Android Studio, https://developer.android.com/studio/publish/app-signing
38. Signing an Android apk with the command line - Random Bits Software Engineering, https://randombits.dev/articles/android/signing-with-cmd
39. Code sign for Android - Power Apps - Microsoft Learn, https://learn.microsoft.com/en-us/power-apps/maker/common/wrap/code-sign-android
40. apksigner | Android Studio, https://developer.android.com/tools/apksigner
41. How to sign an apk through command line - Stack Overflow,

https://stackoverflow.com/questions/50705658/how-to-sign-an-apk-through-command-line
42. Sign an apk using keytool - android studio - Stack Overflow, https://stackoverflow.com/questions/63248036/sign-an-apk-using-keytool
43. Google Play services and runtime permissions, https://developers.google.com/android/guides/permissions
44. Request permission to access nearby Wi-Fi devices | Connectivity - Android Developers, https://developer.android.com/develop/connectivity/wifi/wifi-permissions
45. Inconsistent documentation for Android permission ACCESS_FINE_LOCATION, https://stackoverflow.com/questions/77948674/inconsistent-documentation-for-android-permission-access-fine-location
46. Behavior changes: Apps targeting Android 13 or higher, https://developer.android.com/about/versions/13/behavior-changes-13
47. Android Wi-Fi Direct Architecture: From Protocol Implementation to Formal Specification, https://www.ijert.org/android-wi-fi-direct-architecture-from-protocol-implementation-to-formal-specification
48. Creating P2P Connections with Wi-Fi | Android Developers, https://spot.pcc.edu/~mgoodman/developer.android.com/training/connect-devices-wirelessly/wifi-direct.html
49. WifiP2pManager | API reference - Android Developers, https://developer.android.com/reference/kotlin/android/net/wifi/p2p/WifiP2pManager